

Basic (venv, start to finish) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/venv_setup_practice.md` — notes, not a script. Every block below is a terminal command, so there is nothing to `python -run`: keep a Markdown file with each command you ran and the output you got back, and you will have your own record of the sequence the next time you start a project.

Story — `venv_setup_practice.md`: every later document installs packages into a project, so the venv has to be second nature before any real code exists. Writing the steps down once, with what each one prints, makes it something you can repeat without thinking. **If not:** the first import error in a later document would send you guessing about which Python and which packages you're actually using.

Basic Version

Approach 1 — the direct way (macOS / Linux)

Story: create, activate, install, freeze — the four steps in order, on one platform. **If not:** packages would land in your system Python and leak between projects.

```
# → practice/venv_setup_practice.md
python -m venv .venv
source .venv/bin/activate
pip install requests
pip freeze > requirements.txt
cat requirements.txt
```

Expected output (from the final `cat requirements.txt`):

```
certifi==2024.8.30
charset-normalizer==3.3.2
idna==3.8
requests==2.32.3
urllib3==2.2.3
```

The version numbers may differ on your machine — `pip freeze` lists `requests` itself plus the smaller packages it depends on internally (`certifi` , `charset-normalizer` , `idna` , `urllib3`), which is why there are 5 lines instead of 1.

This version is correct and complete for the exercise as stated.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — with the Windows variant

Story: the same four steps, with Windows' different activate command — so the notes work on any teammate's machine. **If not:** a Windows teammate would get stuck on the very first step.

macOS / Linux:

```
# → practice/venv_setup_practice.md
python -m venv .venv
source .venv/bin/activate
pip install requests
pip freeze > requirements.txt
```

Windows (PowerShell):

```
# → practice/venv_setup_practice.md
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install requests
pip freeze > requirements.txt
```

Expected output, either platform, after `pip freeze > requirements.txt` : no console output (it's redirected into the file). Open `requirements.txt` to see the same 5 lines as the Basic Version above.

Approach 2 — verifying each step as you go

Story: check each step worked before moving on — the venv exists, it's active, the package is inside it. **If not:** a step that silently failed would only show up later as a confusing import error.

```
# → practice/venv_setup_practice.md
python -m venv .venv
ls .venv
# bin/  include/  lib/  pyvenv.cfg

source .venv/bin/activate
which python
```

```
# /path/to/your/project/.venv/bin/python

pip install requests
pip show requests
# Name: requests
# Version: 2.32.3
# Summary: Python HTTP for Humans.
# ...

pip freeze > requirements.txt
wc -l requirements.txt
# 5 requirements.txt
```

Difference from Basic: both Intermediate approaches run the exact same 4 commands as Basic — nothing about the actual environment changes. Approach 1 just adds the Windows equivalent, since `source` and `.venv/bin/activate` are Linux/macOS-specific paths. Approach 2 adds a check after each command (`ls`, `which python`, `pip show`, `wc -l`) so you catch a problem — like activation silently failing — immediately, right where it happened, instead of discovering it 3 steps later when `pip freeze` produces an empty or wrong file.

The most common failure: running `source .venv/bin/activate` from the wrong folder (so `.venv/bin/activate` doesn't exist relative to where you are), or forgetting the `source` keyword on macOS/Linux (`.venv/bin/activate` alone won't work in most shells — it needs to be *sourced* into your current shell, not just executed). Check with `pwd` that you're in the project folder first, and check `(.venv)` actually appears in your prompt after activating.

Which one should you actually write? For this exercise, and for any solo script or small project, the Basic Version's 4 commands typed by hand are completely fine — you're not automating anything you don't already understand. Intermediate Approach 2's per-step checks are worth the extra typing the first few times you do this, until the sequence is second nature.